

Laboratory Activity #3

Productionizing a Modular Data Pipeline

Item	Specification
Core Tools for this Laboratory	Python, SQL, PostgreSQL, Git, Docker, Docker Compose, Apache Airflow, CSV, JSON, Parquet
Primary Output	A reproducible, modular, rerun-safe, benchmarked, partitioned, and orchestrated data pipeline
Submission	Git repository + technical evidence + benchmark report + Airflow run evidence

1. Purpose and Alignment

This laboratory operationalizes Module 2 of our course. You will progressively convert source files into a maintainable pipeline, package it in a reproducible environment, transform raw data into staging and curated layers, compare storage formats, partition data, load PostgreSQL safely, and finally orchestrate the same reusable business logic with Apache Airflow.

2. Scenario

You are part of a data engineering team supporting an e-commerce analytics platform. Source systems export customer, product, and order data. Analysts need a curated sales-order-line dataset in PostgreSQL. The current process is a collection of ad hoc scripts, local settings, and manual runs. Your task is to convert it into a reproducible and operational pipeline suitable for scheduled execution.

The design must preserve raw inputs, make transformations traceable, reject invalid records explicitly, support safe reruns, provide evidence about storage trade-offs, and expose operational state through Airflow.

3. Learning Outcomes

- Create an isolated Python environment and reproduce it with pinned dependencies.
- Refactor pipeline logic into cohesive modules with configuration separated from code.
- Use Git commits/branches/tags to make pipeline evolution reviewable.
- Containerize Python and PostgreSQL services and manage them using Docker Compose.
- Implement raw, staging, curated, and quarantine layers with explicit business and quality rules.
- Apply error handling, audit metadata, deterministic record hashes, and rerun-safe PostgreSQL UPSERT logic.
- Compare CSV, JSON Lines, Parquet, and PostgreSQL using controlled measurements rather than assumptions.
- Partition data by year/month and selectively retrieve/load one partition.
- Build an Airflow DAG with schedule, parameters, dependencies, retries, timeouts, failure handling, and observable run history.

4. Provided Laboratory Package (Github)

Path	Purpose
data/source/	Source datasets: customers.csv, products.json, orders.csv
src/	Minimal modular Python package with TODOs
config/settings.yml	Non-secret pipeline settings
.env.example	Environment-variable template for secrets/runtime settings

Dockerfile	Python pipeline image
docker-compose.yml	Python pipeline + PostgreSQL services
docker-compose.airflow.yml	Goal 4 Airflow services
dags/dss150p_pipeline.py	Airflow DAG starter
sql/init/	Database/bootstrap SQL
templates/	Benchmark, data dictionary, and run-evidence templates
docs/DATASET_GUIDE.md	Source descriptions without revealing exact defect locations

5. General Rules

- Do not edit source files to make the pipeline pass. Invalid source records must be handled by rules and quarantine logic.
- Do not hard-code passwords, host names, file-system roots, or environment-specific values in business logic.
- Do not delete failing records silently. Every rejected record must have a traceable reason.
- Do not implement separate transformation logic inside the Airflow DAG. The DAG calls reusable modules/CLI commands.
- Every major goal must end with a Git commit. Use meaningful commit messages.
- All timestamps created by the pipeline must be UTC unless the instructor explicitly changes this requirement.
- Performance results must be reported as measurements from your machine, not universal claims.

6. Prerequisites and Initial Setup

Required locally: Python 3.11+ recommended, Git, Docker Desktop/Engine with Compose v2, at least 6 GB free disk space, and a text editor/IDE. For the last activity here, you should have enough memory available for PostgreSQL plus Airflow services.

```
git init
git add .
git commit -m "chore: initialize DSS150P module 2 lab"
cp .env.example .env
```

Edit .env and replace the training password. Confirm .env remains ignored by Git:

```
git status --short
git check-ignore .env
```

7. Goal 1 - Reproducible Data Engineering Environments

The goal is not merely to make the code run on your laptop; the goal is to make another machine able to reproduce the environment and invoke the same pipeline interfaces.

7.1 Task A - Build and verify a virtual environment

```
python -m venv .env
# macOS/Linux
source .env/bin/activate
# Windows PowerShell
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
python -m src.cli validate-env
```

Record the Python version and selected package versions. Explain why the virtual environment should not be committed to Git.

7.2 Task B - Evaluate and modularize the pipeline structure

Inspect `src/`. Each module must have one primary responsibility. Complete the package so the command-line interface coordinates extraction, transformation, loading, validation, benchmarking, and partition loading without embedding all logic in one file.

Module	Responsibility	Must NOT contain
<code>src/extract/</code>	Acquire/copy source snapshots into raw layer	Business calculations
<code>src/transform/</code>	Staging cleanup and curated business rules	Airflow-specific code
<code>src/load/</code>	PostgreSQL persistence/upsert	Source-specific cleaning
<code>src/validate/</code>	Data/contract assertions	Transformation side effects
<code>src/benchmark/</code>	Format materialization and timing	Production business logic
<code>src/cli.py</code>	Thin composition/entry points	Duplicate transformation implementations

7.3 Task C - Externalize configuration and secrets

Use `config/settings.yml` for non-secret defaults and `.env` for environment-specific values. `src/config.py` should be the only common place translating these into usable settings. No password may appear in committed Python, YAML, SQL, or DAG files.

```
git grep -n "change_me" -- ":(exclude).env.example" || true
git status --short
```

7.4 Task D - Dockerfile and Docker Compose

Build the pipeline image, start PostgreSQL, and execute the same CLI inside a container. The containerized command should use the same source tree and configuration contract as the local command.

```
docker compose build pipeline
docker compose up -d postgres
docker compose ps
docker compose run --rm pipeline python -m src.cli validate-env
```

Inspect the PostgreSQL initialization. Verify schemas/tables exist:

```
docker exec -it dss150p-postgres psql -U dss150p -d dss150p -c "\dn"
docker exec -it dss150p-postgres psql -U dss150p -d dss150p -c "\dt curated.*"
```

7.5 Task E - Git workflow checkpoint

```
git switch -c goal1-reproducible-environment
git add .
git commit -m "feat: add reproducible pipeline environment"
git log --oneline --decorate -5
```

Evidence required: environment output, Docker/Compose status, Git log, and a short explanation of how configuration is separated from code.

7.6 Goal 1 acceptance tests

- Fresh virtual environment installs from `requirements.txt`.
- `python -m src.cli validate-env` succeeds locally.
- `docker compose run --rm pipeline python -m src.cli validate-env` succeeds.
- PostgreSQL starts healthy and contains the expected schemas.
- `.env` is not tracked by Git.
- Code is divided into modules with clear responsibilities.

8. Goal 2 - ETL/ELT and Transformation Pipeline Development

Build the raw -> staging -> curated pipeline. The raw layer must preserve source snapshots; staging is responsible for technical cleanup and typing; curated applies cross-source business logic and creates analysis-ready measures.

8.1 Required data-layer semantics

Layer	Purpose	Examples
Raw	Reproducible source snapshot; minimal/no business transformation	run-specific copies of source files
Staging	Typing, normalization, source-level deduplication and validity checks	normalized customer email/city; flattened product category; parsed order timestamps
Curated	Cross-source joins, business calculations, consumer-oriented shape	sales order lines with customer/product attributes and monetary measures
Quarantine	Invalid records retained with reason for diagnosis/reprocessing	invalid quantity/status, negative price, orphan foreign key

8.2 Task A - Raw extraction

Implement `extract_sources(run_id)`. It must create a run-specific raw folder such as `data/raw/run_id=run_...` and copy all three source files without altering their content. This creates a reproducible input snapshot.

8.3 Task B - Staging transformations

Implement the following rules. Keep the most recent version of duplicate business keys based on `updated_at`.

Dataset	Required staging rules
customers	<code>customer_id</code> as business key; keep latest <code>updated_at</code> ; lowercase/trim email; trim/title-case city; parse timestamps UTC; retain missing email as a visible quality condition.
products	<code>product_id</code> key; flatten <code>category.name</code> and <code>category.department</code> ; parse price numeric; quarantine negative/invalid prices; keep latest <code>updated_at</code> .
orders	<code>order_id</code> key; parse <code>order_timestamp/updated_at</code> UTC; quantity numeric and 1..20; status must be allowed; keep latest <code>updated_at</code> ; invalid technical records quarantined.

Every staging dataset must add `pipeline_run_id` and `staged_at_utc`. Write staging datasets in Parquet so the next Goal can reuse them.

8.4 Task C - Curated transformation

Join valid orders to valid customers and products. Do not silently drop orphan references. Send them to quarantine with a reason. Calculate:

```
gross_amount    = quantity * unit_price
discount_amount = gross_amount * discount_pct
net_amount      = gross_amount - discount_amount
```

Required curated audit columns: `source_updated_at`, `pipeline_run_id`, `processed_at_utc`, and `record_hash`. `record_hash` should be deterministic for the business content of the row and should not change merely because the pipeline ran at a different time.

8.5 Task D - Error handling

Wrap top-level pipeline stages with meaningful exceptions/logging. A failure must identify the stage and preserve enough context for diagnosis. Do not use broad ``except: pass``. Quarantine is for data-quality exceptions; thrown exceptions are for pipeline/system failures.

8.6 Task E - Rerun-safe PostgreSQL loading

Implement UPSERT into `curated.sales_order_lines` using `order_id` as the conflict key. Use `record_hash` to avoid unnecessary updates when the incoming business content is unchanged. Run the load twice.

```
python -m src.cli run-all
python -m src.cli load
python -m src.cli load
```

Verify no duplicate `order_id` values are created:

```
docker exec -it dss150p-postgres psql -U dss150p -d dss150p -c "SELECT COUNT(*) total, COUNT(DISTINCT order_id)
distinct_orders FROM curated.sales_order_lines;"
```

8.7 Goal 2 acceptance tests

- Raw snapshots are run-specific and source files remain unchanged.
- Duplicate business keys are resolved deterministically using latest updated_at.
- Invalid technical records and orphan references are quarantined with reasons.
- Curated amounts are calculated and audit columns are populated.
- Validation detects duplicate/null business keys and invalid amounts/statuses.
- Repeated load does not create duplicate order_id values.

9. Goal 3 - Storage Systems, File Formats, and Data Organization

Use the same curated logical dataset for all storage comparisons. The objective is to measure trade-offs under a controlled experiment, not to declare one format universally superior.

9.1 Task A - Materialize four storage representations

Write the curated dataset as CSV, JSON Lines, Parquet with compression, and PostgreSQL. For file formats, capture actual file size in bytes. PostgreSQL is a server system, so do not compare its storage footprint using a single file-size number unless you explicitly measure table/index size with a PostgreSQL function.

9.2 Task B - Benchmark methodology

For each format/system measure at minimum: write time, full read/retrieval time, filtered read/query time for status=DELIVERED, and resulting row count. Execute reads at least five times and report the median. Record hardware/OS context because results are machine-dependent.

Metric	CSV	JSON	Parquet	PostgreSQL
Size (bytes)	Measure	Measure	Measure	N/A or documented server table size
Write time	Measure	Measure	Measure	Already loaded / measure separately
Full read/query median	5 runs	5 runs	5 runs	5 queries
Filtered read/query median	status filter	status filter	status filter	WHERE status=...
Interpretation	Discuss parsing/typing	Discuss verbosity	Discuss columnar/compression	Discuss indexes/query engine/concurrency

9.3 Task C - Partitioned Parquet

Add order_year and order_month derived from order_timestamp, then write a partitioned Parquet dataset using both columns. Inspect the resulting directory hierarchy. It should conceptually resemble:

```
data/partitioned/
  order_year=2025/
    order_month=1/
    order_month=2/
  order_year=2026/
    order_month=1/
  ...
```

Read only one selected partition and verify its rows all belong to that year/month. Explain how partitioning can reduce unnecessary I/O when queries filter on partition keys.

9.4 Task D - Load a selected partition to PostgreSQL

```
python -m src.cli load-partition --year 2026 --month 1
```

Record the selected partition in `audit.partition_loads`. Rerun the same partition load and demonstrate that business rows remain deduplicated.

9.5 Goal 3 analysis questions

1. Which file format was smallest on your machine, and what encoding/compression characteristics help explain the result?
2. Which representation was fastest for a full dataset read? Does that imply it is best for every workload?
3. How did filtered retrieval differ between Parquet and PostgreSQL? What additional PostgreSQL design (such as an index) could change the result?
4. Why is JSON Lines generally more pipeline-friendly than one giant JSON array for append/stream-oriented processing?
5. What happens if a partition key has extremely high cardinality or poor query locality?

9.6 Goal 3 acceptance tests

- CSV, JSONL, and Parquet represent the same logical row set.
- Benchmark uses repeated measurements and reports medians.
- Results include file size where meaningful and do not equate size alone with quality.
- Partitioned Parquet is organized by year/month.
- A selected partition can be loaded and audited in PostgreSQL.

10. Goal 4 - Workflow Orchestration and Scheduling with Apache Airflow

Airflow must coordinate the modular pipeline rather than become the location of business logic. The DAG should expose execution order, retries, parameters, state, and failure/recovery evidence.

10.1 Task A - Initialize Airflow

```
docker compose -f docker-compose.yml -f docker-compose.airflow.yml up airflow-init
docker compose -f docker-compose.yml -f docker-compose.airflow.yml up -d airflow-webserver airflow-scheduler
docker compose -f docker-compose.yml -f docker-compose.airflow.yml ps
```

Open `http://localhost:8080`. Training credentials in the starter compose are `admin/admin`; use only for this local laboratory environment.

10.2 Task B - Complete DAG operational configuration

Requirement	Minimum implementation
Schedule	Daily cron schedule such as <code>0 2 * * *</code> ; explain why the chosen cadence is appropriate.
Parameters	<code>run_mode</code> plus selected year/month for partition mode.
Dependencies	<code>extract -> transform -> load -> validate</code> .
Retries	At least 2 retries with an explicit retry delay.
Timeout	Set an execution timeout so a task cannot hang forever.
Failure handling	Callback or equivalent writes/prints task/run/error context.
Catch-up behavior	Explicitly set and explain catchup choice.
Business logic separation	DAG invokes <code>src.cli/module</code> entry points; transformation rules stay in <code>src/</code> .
Run identity	Pass one Airflow run ID through all tasks so <code>pipeline_run_id</code> is consistent across a DAG run.

10.3 Task C - Run and inspect execution states

Trigger a manual full run. Inspect Grid/Graph view, task logs, start/end times, retries, and final state. Capture evidence showing all four tasks and their dependencies.

10.4 Task D - Parameterized partition run

Trigger a DAG run using `run_mode=partition` and a valid year/month. The load task must call the selected-partition loading command using the DAG parameters. Verify `audit.partition_loads` afterward.

10.5 Task E - Deliberate failure and recovery

Create one controlled failure without corrupting source data. Recommended approach: temporarily rename data/source/orders.csv before triggering the DAG, or temporarily set an invalid source path in a noncommitted local configuration. Observe retry attempts and failed state. Restore the source/configuration, then clear/re-run the failed task or trigger a new run.

Required evidence: failed task log, retry attempts, failure callback output, and successful recovery. Explain which steps are safe to rerun and why.

10.6 Optional challenge - backfill reasoning

Explain how you would backfill a historical month if the DAG normally runs daily. You may demonstrate an Airflow backfill/test command if supported by your local version, but the required deliverable is a correct explanation of data interval, idempotency, and avoiding double loads.

10.7 Goal 4 acceptance tests

- Airflow imports the DAG without parse errors.
- DAG has explicit schedule, parameters, dependencies, retries, timeout, and catchup behavior.
- Full and partition-mode runs can be observed in Airflow.
- Controlled failure produces visible retries/failure handling.
- Recovery succeeds without manual database cleanup or duplicate business rows.
- DAG code delegates actual pipeline logic to reusable modules/CLI.
- One pipeline_run_id is propagated consistently across tasks in the same DAG run.

11. Integrated Technical Acceptance Test

Before submission, execute the following sequence from a clean or documented state. Your repository should make these operations understandable to another student or engineer.

```
# Goal 1 environment
python -m src.cli validate-env
docker compose up -d postgres

# Goal 2 full pipeline
python -m src.cli run-all
python -m src.cli load
python -m src.cli validate

# Goal 3 benchmark and partition
python -m src.cli benchmark --repeats 5
python -m src.cli load-partition --year 2026 --month 1

# Goal 4 Airflow
docker compose -f docker-compose.yml -f docker-compose.airflow.yml up -d airflow-webserver airflow-scheduler
```

12. Required Submission

Deliverable	Required evidence/content
Git repository	Meaningful history, .gitignore, no secrets, modular src/, DAG, Compose files, SQL, README.
Goal 1 evidence	venv/package versions, Git log, Docker build/ps, external configuration explanation.
Goal 2 evidence	raw/staging/curated/quarantine counts; sample audit columns; rerun-safe query result; error-handling evidence.
Goal 3 evidence	benchmark_results.csv plus 1-2 page interpretation; partition tree screenshot/listing; selected partition query/load evidence.
Goal 4 evidence	DAG graph/grid, successful full run, parameterized

	partition run, deliberate failure/retries, recovery run.
Technical reflection	Short explanation of modularity, idempotency, storage trade-offs, and orchestration vs business logic.

13. Repository End State (You may extend)

```

project/
├── .env.example
├── .gitignore
├── Dockerfile
├── Dockerfile.airflow
├── docker-compose.yml
├── docker-compose.airflow.yml
├── requirements.txt
├── config/settings.yml
├── data/
│   ├── source/
│   ├── raw/
│   ├── staging/
│   ├── curated/
│   ├── quarantine/
│   ├── benchmarks/
│   └── partitioned/
├── src/
│   ├── extract/
│   ├── transform/
│   ├── load/
│   ├── validate/
│   └── benchmark/
├── dags/dss150p_pipeline.py
├── sql/init/
├── tests/
└── docs/

```

14. Troubleshooting Guide

Symptom	Likely cause / first checks
PostgreSQL container unhealthy	Check .env credentials, port 5432 conflict, docker compose logs postgres; remove volume only if instructor permits reset.
Pipeline cannot connect from host	Inside Docker host is postgres; from local host it is usually localhost. Do not hard-code one value for both contexts.
Parquet import error	Confirm pyarrow installed in active venv/container and correct Python interpreter selected.
Permission error in Airflow logs	Check AIRFLOW_UID, mounted folder ownership, Docker Desktop file-sharing permissions.
DAG not visible	Check scheduler/webserver logs and DAG parse errors; confirm file is mounted under /opt/airflow/dags.
Duplicate rows after rerun	Inspect business key, conflict target, dedup-before-load logic, and whether record_hash includes volatile audit timestamps.
Benchmark results fluctuate	Close competing processes; run multiple repetitions; use median; record environment limitations.

15. Technical Questions

1. Why is `record\_hash` useful for rerun-safe loading, and which columns should not be included in it?
2. Why should raw data usually be preserved even when staging/curated outputs are sufficient for analytics?

3. What is the difference between a data-quality rejection and a system exception?
4. Why might Parquet outperform CSV for selected analytical workloads even if both contain the same rows?
5. Why is a DAG that contains all transformation logic directly considered harder to maintain?
6. How do retries interact with idempotency? Give an example where retries without idempotency cause damage.
7. What trade-off is introduced by partitioning too aggressively?
8. How would you adapt the pipeline if the source became an API or database instead of local files?

16. AI Tool Use and Academic Integrity

Generative AI may be used only within the course policy: as a brainstorming, explanation, debugging, or writing-refinement aid. Students remain responsible for understanding and defending every submitted line of code and design decision. AI-generated code/content cannot be submitted as original work without acknowledgment. The instructor may ask students to explain, modify, or reproduce key pipeline behavior during validation.

17. Final Submission Checklist

Checklist A	Checklist B
☐ No .env/secrets committed.	☐ All required commands documented in README.
☐ Source files unchanged.	☐ Staging/curated/quarantine outputs reproducible.
☐ Rerun-safe PostgreSQL load verified.	☐ Benchmark results and interpretation included.
☐ Partitioned Parquet and selected-partition load verified.	☐ Airflow full/partition/failure/recovery evidence included.
☐ Git history includes Goal 1-4 checkpoints.	☐ Repository runs without relying on undocumented manual edits.